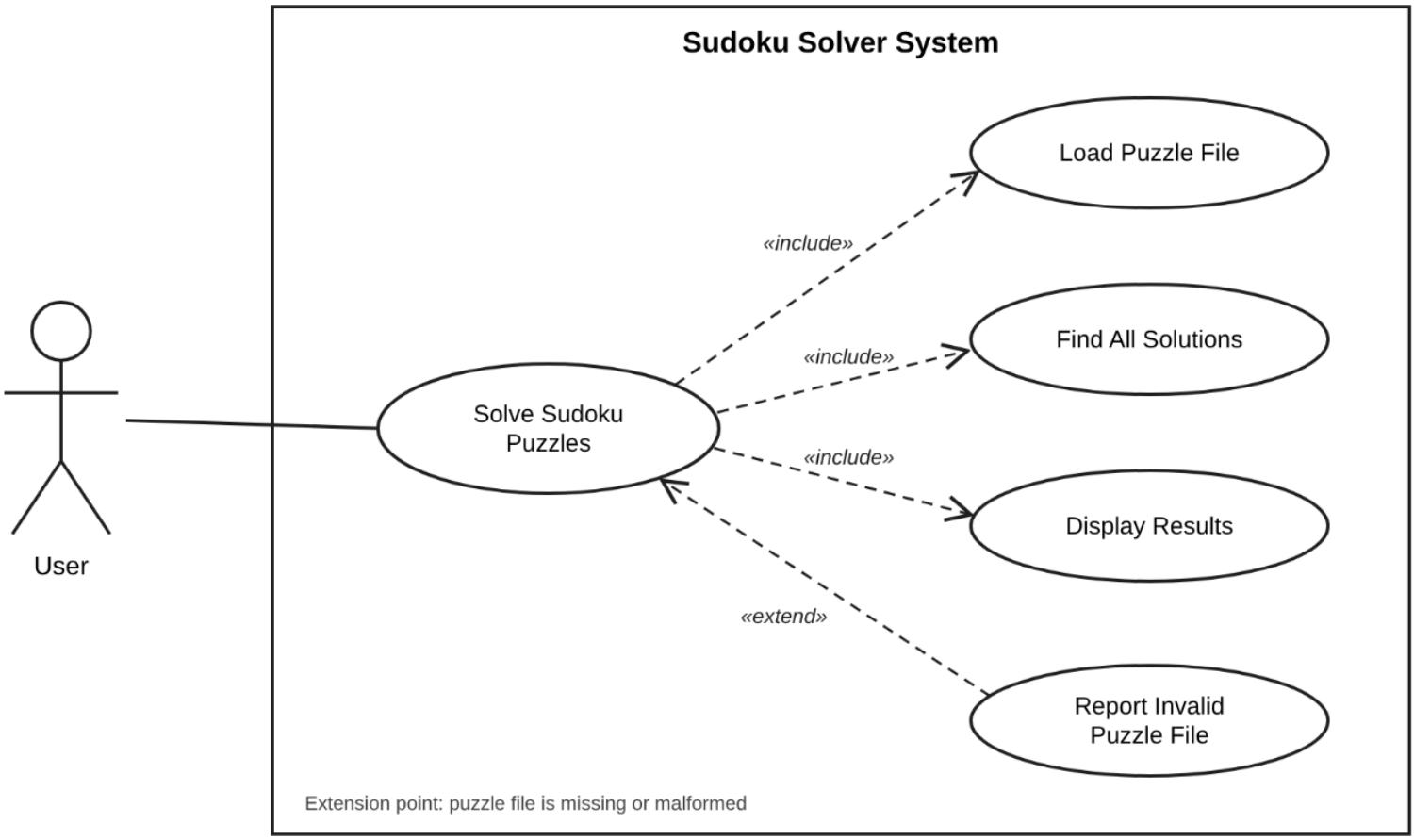


Made by Claude Opus 5.5 (Medium)

Use a GenAI of your choice to help you create a UML Use Case Diagram for this assignment with the level of detail given in the first UML Use Case Modeling lecture:

a. **Identify/discover a use case**

Solve Sudoku puzzles. The user runs the program to solve one or more Sudoku puzzle files.

b. **Provide a brief description for the use case**

The user runs the Sudoku Solver and gives it puzzle files on the command line, or uses the five default files. For each file, the system reads and checks the puzzle, then prints it. It uses a recursive depth-first search with backtracking to find every solution and prints each one. If there is no solution, it prints "No solution found." If a file is missing or not a valid 9x9 puzzle, the system reports an error and moves on to the next file.

c. **Provide a step-by-step flow (outline) of the events for the use case**

Step 1: The user starts the program, either with a list of puzzle files or with none.
Step 2: If no files are given, the system uses puzzle1.txt through puzzle5.txt.
Step 3: For each file:

i. The system prints the file name.
ii. The system opens and reads the file (Load Puzzle File).
iii. The system checks that the file has exactly 81 cells, each a digit 1–9 or an underscore (_).
iv. The system checks that no given digits conflict.
v. The system prints the puzzle as read.
vi. The system searches for every solution (Find All Solutions).
vii. The system prints each solution as it is found, then the total count. If there are none, it prints “No solution found” (Display Results).

Step 4: The program ends after the last file.

Alternate flows:

- At step 3.ii, if the file can’t be opened, the system prints an error and skips to the next file (Report Invalid Puzzle File).
- At step 3.iii, if the file has a bad character or the wrong number of cells, the system prints an error and skips to the next file (Report Invalid Puzzle File).
- At step 3.iv, if given digits conflict, the system prints the puzzle and “No solution found,” then skips the search.

d. Provide a detailed description for the use case

The Solve Sudoku Puzzles use case begins when the User, a person running the program from the terminal, starts the Sudoku Solver. Before it starts, the program must be compiled and the puzzle files must be in the working directory or given by path. The user may list puzzle files on the command line; if none are given, the system uses puzzle1.txt through puzzle5.txt. For each file, the system prints the file name and loads the puzzle (Load Puzzle File). It checks that the file has exactly 81 cells, each a digit from 1 to 9 or an underscore marking a blank, and that no given digits conflict in any row, column, or 3x3 block. It then prints the original puzzle and runs a recursive depth-first search with backtracking (Find All Solutions). At each step, the search fills the empty cell with the fewest candidates, tries each legal digit, recurses, and backtracks when it reaches a dead end. Each time all 81 cells are filled, the system prints that solution with its number, and when the search ends it prints "Solutions found: N," or "No solution found" if there are none (Display Results). If a puzzle has more than one solution, every one is printed. Several alternate flows can occur. If a file cannot be opened, the system prints "Error: could not open" with the file name. If a file has an invalid character or the wrong number of cells, it prints that the file is not a valid 9x9 puzzle. Both errors are handled by Report Invalid Puzzle File, and the system moves on to the next file. If the given digits conflict, the system prints the puzzle and "No solution found" without searching. The use case ends when every file has a report, either an error message or the puzzle followed by its solutions or "No solution found." The puzzle files themselves are not changed. The system assumes each puzzle is a standard 9x9 Sudoku with blanks marked by underscores, and it ignores whitespace between cells.